

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Naila Atresia Rahma

NIM. 2510817120014

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Naila Atresia Rahma
NIM : 2510817120014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
KONVERSI ADJACENCY LIST KE ADJACENCY MATRIX	1
A. Source Code.....	1
B. Output Program	2
C. Pembahasan	2
KONVERSI ADJACENCY MATRIX KE ADJACENCY LIST	4
A. Source Code.....	4
B. Output Program	5
C. Pembahasan	5
BFS : JARAK TERPENDEK KELUAR LABIRIN	7
A. Source Code.....	7
B. Output Program	8
C. Pembahasan	8
DFS: MENGHITUNG SEMUA JALUR KELUAR LABIRIN.....	10
A. Source Code.....	10
B. Output Program	11
A. Pembahasan	11
TREE TRAVERSAL.....	13
A. Source Code.....	13
B. Output Program	15

C. Pembahasan	15
TAUTAN GIT	17

DAFTAR GAMBAR

Gambar 1. Screenshot hasil Konversi Adjacency List Ke Adjacency Matrix.....	2
Gambar 2. Screenshot hasil Konversi Adjacency Matrix ke Adjacency List	5
Gambar 3. Screenshot hasil Bfs : Jarak Terpendek Keluar Labirin	8
Gambar 4. Screenshot hasil DFS: Menghitung Semua Jalur Keluar Labirin	11
Gambar 5. Screenshot hasil Tree Traversal.....	15

DAFTAR TABEL

Tabel 1. Source Code AdjacencyList.CPP	1
Tabel 2. Source Code AdjacencyMatrix.cpp.....	4
Tabel 3. Source Code BFS.cpp.....	7
Tabel 4. Source Code DFS.cpp	10
Tabel 5. Source Code Main.cpp	13

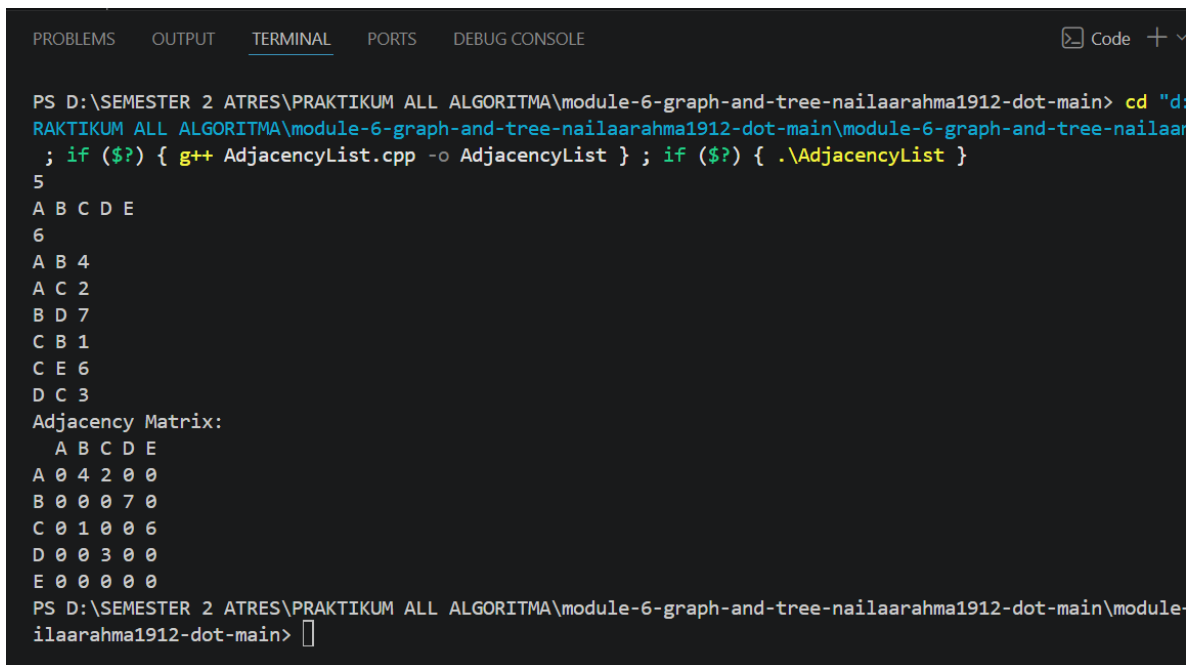
KONVERSI ADJACENCY LIST KE ADJACENCY MATRIX

A. Source Code

Tabel 1. Source Code AdjacencyList.CPP

```
1  #include <iostream>
2  #include <vector>
3  #include <map>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9      vector<char> vertex(N);
10     map<char,int> idx;
11
12     for (int i = 0; i < N; i++) {
13         cin >> vertex[i];
14         idx[vertex[i]] = i;
15     }
16     int M;
17     cin >> M;
18     vector<vector<int>> matrix(N, vector<int>(N, 0));
19     for (int i = 0; i < M; i++) {
20         char u, v;
21         int w;
22         cin >> u >> v >> w;
23         matrix[idx[u]][idx[v]] = w;
24     }
25     cout << "Adjacency Matrix:\n";
26     cout << " ";
27     for (char c : vertex)
28         cout << c << " ";
29     cout << '\n';
30     for (int i = 0; i < N; i++) {
31         cout << vertex[i] << " ";
32         for (int j = 0; j < N; j++) {
33             cout << matrix[i][j] << " ";
34         }
35         cout << '\n';
36     }
37     return 0;
38 }
```

B. Output Program



```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main> cd "d:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-graph-and-tree-nailaarahma1912-dot-main"
; if ($?) { g++ AdjacencyList.cpp -o AdjacencyList } ; if ($?) { .\AdjacencyList }
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-graph-and-tree-nailaarahma1912-dot-main>

```

Gambar 1. Screenshot hasil Konversi Adjacency List Ke Adjacency Matrix

C. Pembahasan

Pertama, algoritma ini bertujuan untuk memetakan hubungan antar titik node/vertex dalam sebuah graf ke dalam bentuk matriks dua dimensi. Karena grafnya ini directed dan weighted matriks ini menyimpan informasi "Dari titik A ke titik B ada bobot sebesar X". Kalau tidak ada hubungan itu nilainya 0.

Teknik yang dipakai itu Mapping Index, karena nama titiknya bisa karakter misalnya A, B, C, tidak bisa langsung pakai karakter tersebut sebagai indeks array. Cara kerjanya pakai `std::map` untuk mengaitkan karakter dengan indeks angka (0, 1, 2, ...), lalu minta input jumlah titik dan nama titiknya, habistu masukkan nama titik itu ke map supaya tau A itu indeks 0, B itu indeks 1, dan seterusnya, lalu buat matriks 2D vektor di dalam vector yang diinisialisasi dengan angka 0, habistu baca input sisi edge dan bobotnya, lalu masukkan ke matriks sesuai posisi indeks yang sudah kita *map* tadi.

Asumsi yang dibuat tu graf berarah, hubungan dari A ke B tidak sama dengan B ke A seperti terlihat yg ada di output, A ke B ada nilai, tapi B ke A tidak. Lalu Bobot, bobot

dianggap positif kalau ada hubungan yg tidak didefinisikan dalam inputnya, otomatis nilainya 0.

Analisis Kompleksitas :

1. Time Complexity (Waktu):

- Proses mapping vertex : $O(N \log N)$ karena pakai `std::map`
- Proses mengisi matriks : $O(M \log N)$ di mana si M nya itu tu jumlah edge, karena setiap edge perlu akses map untuk mencari indeks
- Proses cetak : $O(N^2)$
- Total : $O((N+M) \log N + N^2)$, yg didominasi oleh $O(N^2)$ kalau N nya lebih besar dari M.

2. Space Complexity (Ruang):

- Penyimpanan matriks : $O(N^2)$
- Penyimpanan map dan list vertex : $O(N)$
- Total : $O(N^2)$

Proof Pembuktian nya :

- Inputnya: 5 titik (A, B, C, D, E) dan 6 sisi,
- Mapping: A=0, B=1, C=2, D=3, E=4,
- Eksekusinya : Input "A B 4": `matrix[idx['A']][idx['B']] = 4` jadi `matrix[0][1] = 4`.

Input "C B 1": `matrix[idx['C']][idx['B']] = 1` jadi `matrix[2][1] = 1`. Dan seterusnya..

- Hasil: Di baris pertama index 0 atau titik A, kolom kedua index 1 atau titik B bernilai 4. Jadi ini sama dengan output A 0 4 2 0 0. Terbukti kalau nilai di `matrix[i][j]` ya memang mencerminkan hubungan langsung dari titik ke-i ke titik ke-j.

KONVERSI ADJACENCY MATRIX KE ADJACENCY LIST

A. Source Code

Tabel 2. Source Code AdjacencyMatrix.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int N;
7      cin >> N;
8      vector<char> vertex(N);
9      for (int i = 0; i < N; i++)
10         cin >> vertex[i];
11     vector<vector<int>> matrix(N, vector<int>(N));
12     for (int i = 0; i < N; i++)
13         for (int j = 0; j < N; j++)
14             cin >> matrix[i][j];
15     cout << "Adjacency List:\n";
16     for (int i = 0; i < N; i++) {
17         cout << vertex[i] << " -> ";
18         bool first = true;
19         for (int j = 0; j < N; j++) {
20             if (matrix[i][j] > 0) {
21                 if (!first)
22                     cout << ", ";
23                 cout << "("
24                     << vertex[j]
25                     << ", "
26                     << matrix[i][j]
27                     << ")";
28                 first = false;
29             }
30         }
31         if (first)
32             cout << "-";
33         cout << '\n';
34     }
35     return 0;
36 }
```

B. Output Program

```

PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-graph-and-tree-nailaarahma1912-dot-main> cd "d:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\" ; if ($?) { g++ AdjacencyMatrix.cpp -o AdjacencyMatrix } ; if ($?) { ./AdjacencyMatrix }
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-graph-and-tree-nailaarahma1912-dot-main>

```

Gambar 2. Screenshot hasil Konversi Adjacency Matrix ke Adjacency List

C. Pembahasan

Algoritma ini untuk mengubah representasi graf dari format Adjacency Matrix menjadi Adjacency List. Fokus nya tu buat membaca bobot hubungan dari matriks dan menyajikannya dalam format teks yang lebih mudah dibaca, disetiap titik menampilkan daftar tujuan serta bobot sisi yang terhubung dengannya.

Teknik nya yaitu Nested Iteration atau perulangan bersarang untuk memindai setiap sel dalam matriks. Cara kerjanya itu program membaca data matriks berukuran $N \times N$ terlebih dahulu, lalu habistu setelah matriksnya terisi, program melakukan perulangan untuk setiap baris. Di dalam setiap baris, programnya kembali melakukan perulangan untuk memeriksa kolom ketitik tujuan. kalau nilai dalam `matrix[i][j]` lebih besar dari 0, artinya terdapat sisi yang valid, jadi programnya akan mencetak nama titik tujuan dan bobotnya. Variabel `first` digunakan untuk memformat tampilan agar tanda koma hanya muncul di antara elemen daftar bukannya di depan elemen pertama.

Asumsi yang dibuat tu Representasi Sisi, naa nilai 0 di dalam matriks dianggap sebagai tidak adanya hubungan, sedangkan nilai di atas 0 itu merepresentasikan adanya sisi berbobot. Struktur Graf, graf diasumsikan sebagai graf berarah dengan bobot setiap nilai di `matrix[i][j]` adalah bobot dari titik `i` ke titik `j`. Input, jumlah titik `N` dan label titik dalam bentuk karakter telah diketahui dan diurutkan sesuai dengan indeks matriks.

Analisis Kompleksitas :

- Time Complexity (Waktu):
- $O(N^2)$, karena program harus memeriksa setiap sel dalam matriks $N \times N$ untuk menentukan apakah ada koneksi yang perlu dicetak.
- Space Complexity (Ruang):
 $O(N^2)$ untuk menyimpan matriks dalam `vector<vector<int>>` dan
 $O(N)$ untuk menyimpan label titik jadi total kebutuhan ruang adalah $O(N^2)$

Proof Pembuktiannya :

Naa di output nya matriks dengan input A, B, C, D, E. Untuk baris pertama titik A, program memeriksa kolom kolomnya `matrix[0][1]` bernilai 4 dan `matrix[0][2]` bernilai 2. Programnya mendeteksi nilai > 0 pada indeks 1 dan 2, sehingga mencetak A $\rightarrow$ (B, 4) , (C, 2) . Begitu juga di titik E baris kode ke 5, karena semua kolom bernilai 0, kondisi `if (first)` tetap benar hingga akhir, jadi menghasilkan output E $\rightarrow$ - . ini membuktikan bahwa algoritma berhasil memproses matriks secara sistematis dan melakukan pemetaan hubungan antar titik dengan akurat sesuai logika graf yang didefinisikan.

BFS : JARAK TERPENDEK KELUAR LABIRIN

A. Source Code

Tabel 3. Source Code BFS.cpp

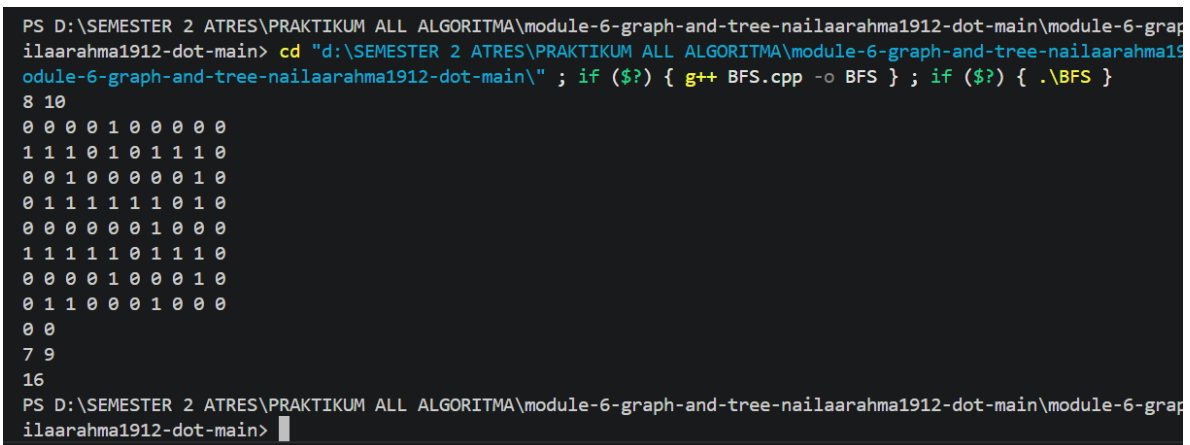
```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  struct Node {
7      int r;
8      int c;
9      int dist;
10 };
11 int main() {
12     int R, C;
13     cin >> R >> C;
14     vector<vector<int>> grid(R, vector<int>(C));
15     for (int i = 0; i < R; i++)
16         for (int j = 0; j < C; j++)
17             cin >> grid[i][j];
18     int sr, sc;
19     int fr, fc;
20     cin >> sr >> sc;
21     cin >> fr >> fc;
22     vector<vector<bool>> vis(R, vector<bool>(C, false));
23     queue<Node> q;
24     q.push({sr, sc, 0});
25     vis[sr][sc] = true;
26     int dr[4] = {-1, 1, 0, 0};
27     int dc[4] = {0, 0, -1, 1};
28     while (!q.empty()) {
29         Node cur = q.front();
30         q.pop();
31         if (cur.r == fr && cur.c == fc) {
32             cout << cur.dist << '\n';
33             return 0;
34         }
35         for (int k = 0; k < 4; k++) {
36             int nr = cur.r + dr[k];
37             int nc = cur.c + dc[k];
38             if (nr < 0 || nr >= R || nc < 0 || nc >= C)
39                 continue;
40             if (grid[nr][nc] == 1)
```

```

41         continue;
42         if (vis[nr][nc])
43             continue;
44         vis[nr][nc] = true;
45         q.push({nr, nc, cur.dist + 1});
46     }
47 }
48 cout << -1 << '\n';
49 return 0;
50 }

```

B. Output Program



```

PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-grap
ilaarahma1912-dot-main> cd "d:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma19
odule-6-graph-and-tree-nailaarahma1912-dot-main\" ; if ($?) { g++ BFS.cpp -o BFS } ; if ($?) { .\BFS }
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6-grap
ilaarahma1912-dot-main>

```

Gambar 3. Screenshot hasil Bfs : Jarak Terpendek Keluar Labirin

C. Pembahasan

Algoritma ini tu implementasi Breadth-First Search (BFS) yang digunakan untuk mencari jalur terpendek dari satu titik awal ke titik tujuan dalam sebuah grid dua dimensi. Programnya akan menentukan langkah paling bagus dengan cara menjelajahi grid lapis per lapis buat memastikan bahwa jarak yang ditemukan tu yang paling minimal dalam graf yang tidak memiliki bobot.

Teknik yang digunakan adalah Queue based Traversal dengan state space search. Cara kerjanya tu dimulai dengan memasukkan posisi awal ke dalam queue dan menandainya sebagai sudah dikunjungi visited. Habistu programnya memproses titik dari bagian depan queue satu persatu. Jadi setiap titik tu programnya coba melangkah ke 4 arah atas, bawah, kiri, kanan menggunakan array `dr` dan `dc`. Kalau langkah nya tu berada di dalam batas grid bukan rintangan yg ditandai dengan nilai 1 belum pernah dikunjungi,

posisi baru nya tu ditandai sebagai sudah dikunjungi habistu dimasukkan ke dalam queue dengan nilai jarak yang telah ditambah satu.

Asumsi yang dibuat tu grid dianggap sebagai graf, jadi setiap sel itu node sama koneksi antar sel yang berdekatan adalah edge. Rintangan, nilai 1 pada grid merepresentasikan rintangan yg tidak dapat dilalui sedangkan nilai 0 adalah jalur yang dapat dilewati. Jalur Terpendek, diasumsikan bahwa pencarian jalur paling efisien hanya bisa dilakukan melalui empat arah tadi.

Analisis Kompleksitas :

- Time Complexity (Waktu):
 $O(R \times C)$, si R itu adalah jumlah baris dan C itu adalah jumlah kolomnya. Setiap sel di dalam grid maksimal hanya diproses satu kali dalam antrean.
- Space Complexity (Ruang):
 $O(R \times C)$, ruang ini dibutuhkan untuk menyimpan matriks grid, matriks vis untuk melacak status kunjungan dan queue yang dalam kasus dapat menyimpan proporsional terhadap total sel dalam grid.

Proof Pembuktiannya :

Sesuai dengan contoh input nya, terdapat grid berukuran 8x10 dengan titik awal (0, 0) dan titik tujuan (7, 9) . Programnya memulai penelusuran dari (0, 0) dengan jarak 0. Melalui perulangan `while`, program terus memperluas jangkauan pencarian ke sel tetangga yang bernilai 0. Jadi setiap kali sebuah sel tetangga yang valid ditemukan, sel tersebut didorong ke dalam queue dengan `dist + 1`. Ketika sel (7, 9) berhasil diambil dari queue, program mencetak nilai `dist`-nya, yaitu 16. ini membuktikan kalau algoritma secara konsisten menambah jarak secara bertahap dan menemukan jumlah langkah minimum dari titik awal ke tujuan melalui grid yang penuh rintangan tersebut.

DFS: MENGHITUNG SEMUA JALUR KELUAR LABIRIN

A. Source Code

Tabel 4. Source Code DFS.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int R, C;
6	vector<vector<int>> grid;
7	vector<vector<bool>> visited;
8	int fr, fc;
9	long long totalPath = 0;
10	int dr[4] = {-1, 1, 0, 0};
11	int dc[4] = {0, 0, -1, 1};
12	void dfs(int r, int c) {
13	if (r == fr && c == fc) {
14	totalPath++;
15	return;
16	}
17	visited[r][c] = true;
18	for (int k = 0; k < 4; k++) {
19	int nr = r + dr[k];
20	int nc = c + dc[k];
21	if (nr < 0 nr >= R nc < 0 nc >= C)
22	continue;
23	if (grid[nr][nc] == 1)
24	continue;
25	if (visited[nr][nc])
26	continue;
27	dfs(nr, nc);
28	}
29	visited[r][c] = false;
30	}
31	int main() {
32	cin >> R >> C;
33	grid.assign(R, vector<int>(C));
34	for (int i = 0; i < R; i++)
35	for (int j = 0; j < C; j++)
36	cin >> grid[i][j];
37	int sr, sc;
38	cin >> sr >> sc;


```

39     cin >> fr >> fc;
40     visited.assign(R, vector<bool>(C, false));
41     dfs(sr, sc);
42     cout << totalPath << '\n';
43     return 0;
44 }

```

B. Output Program

```

PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6
ilaarahma1912-dot-main> cd "d:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaara
odule-6-graph-and-tree-nailaarahma1912-dot-main\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunn
{ .\tempCodeRunnerFile }
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-dot-main\module-6
ilaarahma1912-dot-main>

```

Gambar 4. Screenshot hasil DFS: Menghitung Semua Jalur Keluar Labirin

A. Pembahasan

Algoritma ini menggunakan Depth-First Search (DFS) yang bersifat untuk menghitung total kemungkinan jalur dari satu titik awal ke titik tujuan dalam sebuah labirin atau grid dua dimensi. Beda sama BFS yang mencari jalur terpendek, algoritma ini ni menjelajahi setiap cabang jalur yang mungkin hingga mencapai titik akhir.

Teknik nya yaitu backtracking (penelusuran balik), jadi cara kerjanya dimulai dengan memanggil fungsi `dfs` dari koordinat awal. Fungsi nya buat meriksa apakah posisi saat ini adalah titik tujuan, kalau iya variabel `totalPath` akan ditambah satu. kalau belum, program menandai sel sebagai sudah dikunjungi `visited` lalu coba melangkah ke 4 arah atas, bawah, kiri, kanan. Langkah ini hanya dilakukan kalau posisi berikutnya tu valid Setelah semua kemungkinan dari satu arah sdh dicoba, program melakukan backtracking dengan mengubah status sel tersebut menjadi tidak dikunjungi kembali `unvisited` biar sel itu bisa digunakan oleh jalur lain yang berbeda.

Asumsinya tu grid sebagai area jelajah, nilai 0 di grid merepresentasikan jalan yang bisa dilalui, sedangkan kalau nilai 1 adalah rintangan atau dinding. Lalu asumsi pergerakan, nah pergerakan terbatas pada 4 arah tadi atas, bawah, kiri, kanan. Backtracking, jadi setiap sel yang sudah dikunjungi dalam satu jalur harus dibebaskan kembali statusnya jadi saat penelusuran mundur, sehingga sel tersebut memungkinkan untuk menjadi bagian dari jalur alternatif lainnya.

Analisis Kompleksitas :

- Time Complexity (Waktu):

$O(3^{(R \times C)})$, algoritma mengeksplorasi banyak kemungkinan jalur, jumlah percabangan setiap langkah adalah 3 karena tidak boleh kembali ke posisi tepat sebelumnya, jadi yang dipangkatkan dengan total sel dalam grid.

- Space Complexity (Ruang):

$O(R \times C)$, ruang ini digunakan untuk tumpukan rekursi (recursion stack) pada fungsi `dfs` dan penyimpanan status `visited` serta `grid`.

Proof pembuktiannya :

Berdasarkan input soalnya tuh, grid yg berukuran 5x6 dengan titik awal (0, 0) dan tujuan (4, 5). Program nya menelusuri setiap kombinasi langkah yang mungkin, setiap fungsi mencapai koordinat (4, 5), `totalPath` nya bertambah. contohnya kalau ada 4 jalur yang mencapai tujuan maka setelah semua rekursi selesai, variabel `totalPath` nilainya 4. Nah ini membuktikan kalau mekanisme `visited[r][c] = false` di baris 29 sangat krusial biar jalur yang telah terpakai tidak memblokir kemungkinan jalur lain untuk mencapai titik tujuan yang sama.

TREE TRAVERSAL

A. Source Code

Tabel 5. Source Code Main.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include "RedBlackTree.h"
5
6  using namespace std;
7  void preorder(
8      const RedBlackTree::Node* node,
9      const RedBlackTree::Node* nil,
10     vector<int>& result)
11  {
12     if (node == nil) return;
13     result.push_back(node->key);
14     preorder(node->left, nil, result);
15     preorder(node->right, nil, result);
16  }
17  void inorder(
18     const RedBlackTree::Node* node,
19     const RedBlackTree::Node* nil,
20     vector<int>& result)
21  {
22     if (node == nil) return;
23     inorder(node->left, nil, result);
24     result.push_back(node->key);
25     inorder(node->right, nil, result);
26  }
27  void postorder(
28     const RedBlackTree::Node* node,
29     const RedBlackTree::Node* nil,
30     vector<int>& result)
31  {
32     if (node == nil) return;
33     postorder(node->left, nil, result);
34     postorder(node->right, nil, result);
35     result.push_back(node->key);
36  }
37  void printTraversal(
38     const string& name,
39     const vector<int>& values)
40  {
```

```

41     cout << "[" << name << "]" : ";
42     for (size_t i = 0; i < values.size(); i++) {
43         if (i) cout << ' ';
44         cout << values[i];
45     }
46     cout << '\n';
47 }
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51     int N;
52     cin >> N;
53     RedBlackTree tree;
54     for (int i = 0; i < N; i++) {
55         int x;
56         cin >> x;
57         tree.insert(x);
58     }
59     int Q;
60     cin >> Q;
61     if (tree.empty()) {
62         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
63         return 0;
64     }
65     while (Q--) {
66         string query;
67         cin >> query;
68         if (query == "PREORDER") {
69             vector<int> res;
70             preorder(tree.root(), tree.nil(), res);
71             printTraversal("Preorder", res);
72         }
73         else if (query == "INORDER") {
74             vector<int> res;
75             inorder(tree.root(), tree.nil(), res);
76             printTraversal("Inorder", res);
77         }
78         else if (query == "POSTORDER") {
79             vector<int> res;
80             postorder(tree.root(), tree.nil(), res);
81             printTraversal("Postorder", res);
82         }
83         else if (query == "ALL") {
84             vector<int> pre;
85             vector<int> in;
86             vector<int> post;

```

```

87         preorder(tree.root(), tree.nil(), pre);
88         inorder(tree.root(), tree.nil(), in);
89         postorder(tree.root(), tree.nil(), post);
90         printTraversal("Preorder", pre);
91         printTraversal("Inorder", in);
92         printTraversal("Postorder", post);
93     }
94 }
95 return 0;
96 }

```

B. Output Program

```

7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS D:\SEMESTER 2 ATRES\PRAKTIKUM ALL ALGORITMA\module-6-graph-and-tree-nailaarahma1912-
ilaarahma1912-dot-main>

```

Gambar 5. Screenshot hasil Tree Traversal

C. Pembahasan

Algoritma ini adalah implementasi Tree Traversal untuk struktur data Red-Black Tree. Tujuannya itu untuk menampilkan seluruh data dalam tree dengan tiga urutan kunjungan yang berbeda: Preorder (akar, kiri, kanan), Inorder (kiri, akar, kanan), dan Postorder (kiri, kanan, akar).

Teknik yang digunakan adalah rekursi pada struktur data tree. Cara kerjanya dimulai dengan memasukkan data ke dalam Red-Black Tree. Kemudian, fungsi traversal (preorder, inorder, postorder) dipanggil secara rekursif.

Asumsi nya Struktur Data, data disimpan dalam Red-Black Tree yang sudah memiliki fungsi insert, root, dan nil. Lalu asumsi Basis Rekursi, fungsi traversal akan berhenti return jika simpul yang dikunjungi bernilai nil. Habistu asumsi Query, program

mengasumsikan input berupa string perintah "PREORDER", "INORDER", "POSTORDER", atau "ALL" untuk menentukan jenis traversal yang diinginkan.

Analisis Kompleksitas :

- Time Complexity (Waktu):
 $O(N)$, nah N ini tu adalah jumlah simpul dalam tree. jadi Setiap simpul dikunjungi satu kali untuk setiap jenis traversal.
- Space Complexity (Ruang):
 $O(H)$ untuk tumpukan rekursi (recursion stack), dengan H itu adalah tinggi tree. Jadi karna ini ni adalah Red-Black Tree, tinggi pohon dijaga tetap $O(\log N)$, jadi kompleksitas ruang rata rata adalah $O(\log N)$.

Proof pembuktian :

Jadi sesuai output nya itu dengan data (50, 30, 70, 20, 40, 60, 80), Preorder, menampilkan akar 50, lalu dikiri 30, 20, 40, lalu habistu kanan 70, 60, 80. Output: 50 30 20 40 70 60 80. Terus ada Inorder, nah inorder ini menampilkan data dari terkecil ke terbesar karena sifat Binary Search Tree pada Red-Black Tree, Outputnya 20 30 40 50 60 70 80. Lalu ada Postorder, ini ni menampilkan kiri nilainya 20, 40, 30, lalu kanan 60, 80, 70, baru akarnya 50, Outputnya 20 40 30 60 80 70 50. Naa ini udaa sesuai dengan urutan masing masing metode traversal, membuktikan bahwa fungsi rekursif berjalan dengan benar.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-nailaarahma1912-dot>

